



Universidad
Indoamérica

UNIDAD 5 · FUNDAMENTOS DE SISTEMAS OPERATIVOS

Tuberías y composición de procesos

Universidad Tecnológica Indoamérica

Laboratorio TD4

¿Cómo se comunican dos procesos aislados?

Flujos de bytes

Tuberías (esta unidad),
FIFO (unidad 3), sockets
(unidad 8)

Avisos

Señales (unidad 6)

Memoria común

`mmap` y memoria
compartida (unidad 6),
hilos (unidad 7)

La tubería es la más simple y la más usada: cada `|` del shell es una.

Una tubería: `pipe(fds)`

- Crea un búfer en el núcleo y dos descriptores: `fds[0]` para leer y `fds[1]` para escribir
- Un solo sentido, un flujo de bytes sin mensajes ni nombre
- Solo la comparten los procesos que heredan los descriptores: **se crea antes del `fork`**
- Cada proceso **cierra el extremo que no usa**

C tds/td4/mypipe.c

```
exit_if(pipe(fds) == -1, "pipe");
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {
    exit_if(close(fds[1]) == -1, "close"); /* ch.
    char buffer[READ_MAX];
    ssize_t n = read(fds[0], buffer, READ_MAX); /* ne
    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n)
    fprintf(stderr, "I am the child, I read %zd bytes an
    exit_if(close(fds[0]) == -1, "close");
    return EXIT_SUCCESS;
```

Las reglas de una tubería

Leer con la tubería vacía	bloquea... o devuelve 0 si ya no queda ningún escritor
Escribir con la tubería llena	bloquea hasta que alguien lea
Escribir sin ningún lector	señal SIGPIPE (o EPIPE si se ignora)
Escritura $\leq$ PIPE_BUF (4096 B)	atómica: no se mezcla con otras

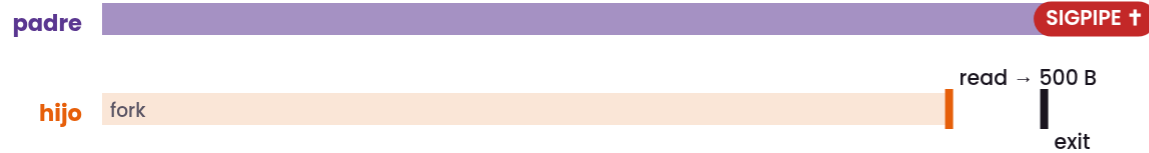
Capacidad por omisión en el equipo del curso:

Salida real

```
$ python3 -c 'import os, fcntl; r, w = os.pipe();  
print(fcntl.fcntl(w, fcntl.F_GETPIPE_SZ), "bytes")'  
65536 bytes
```

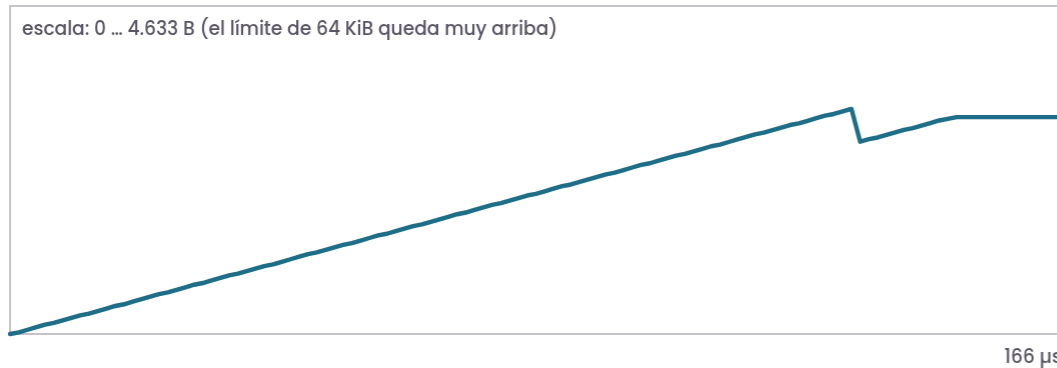
10, 300, 10 000 mensajes: ¿qué pasa?

línea de tiempo (modelo, μs)

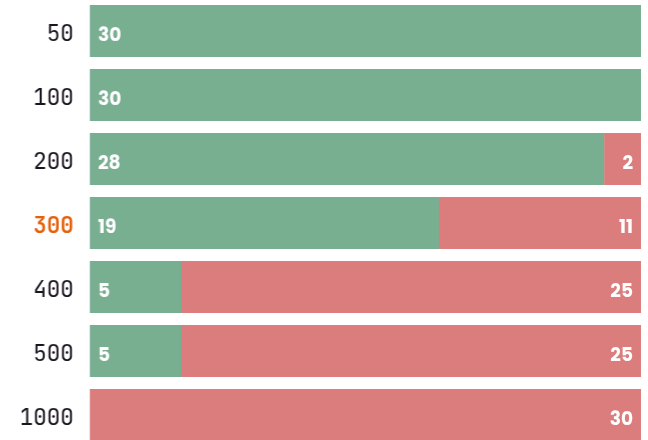


bytes en la tubería

escala: 0 ... 4.633 B (el límite de 64 KiB queda muy arriba)



medición real: 30 ejecuciones por tamaño



verde: estado 0 · rojo: 141 (SIGPIPE)
mensajes

Cerrar o no cerrar `fds[0]` en el padre

Salida real

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"  
I am the child, I read 500 bytes and I leave  
exit status of mypipe: 141
```

Salida real

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by timeout: it was blocked)"  
I am the child, I read 500 bytes and I leave  
exit status: 124 (124 = killed by timeout: it was blocked)
```

Con el extremo cerrado, el padre recibe SIGPIPE (141 = 128 + 13) cuando el hijo se va. Sin cerrarlo, el padre es su propio "lector": llena los 64 KiB y se bloquea para siempre (124 = lo mató `timeout`). Al lanzarlo con `&`, el shell informa el estado de fin.

Conectar un programa ajeno: `dup2` + `exec`

C tds/td4/upper.c

```
/* parent */
exit_if(close(fds[0]) == -1, "close");
exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
exit_if(close(fds[1]) == -1, "close");          /* 1 already closed */
execlp("ps", "ps", (char *)NULL);              /* "ps" to run */
exit_if(1, "execlp ps");
```

Salida real

```
$ ./upper | head -4
  PID TTY          TIME CMD
 6706 ?            00:00:25 UVICORN
 9771 ?            00:00:03 SYSTEMD
 9801 ?            00:00:00 (SD-PAM)
```

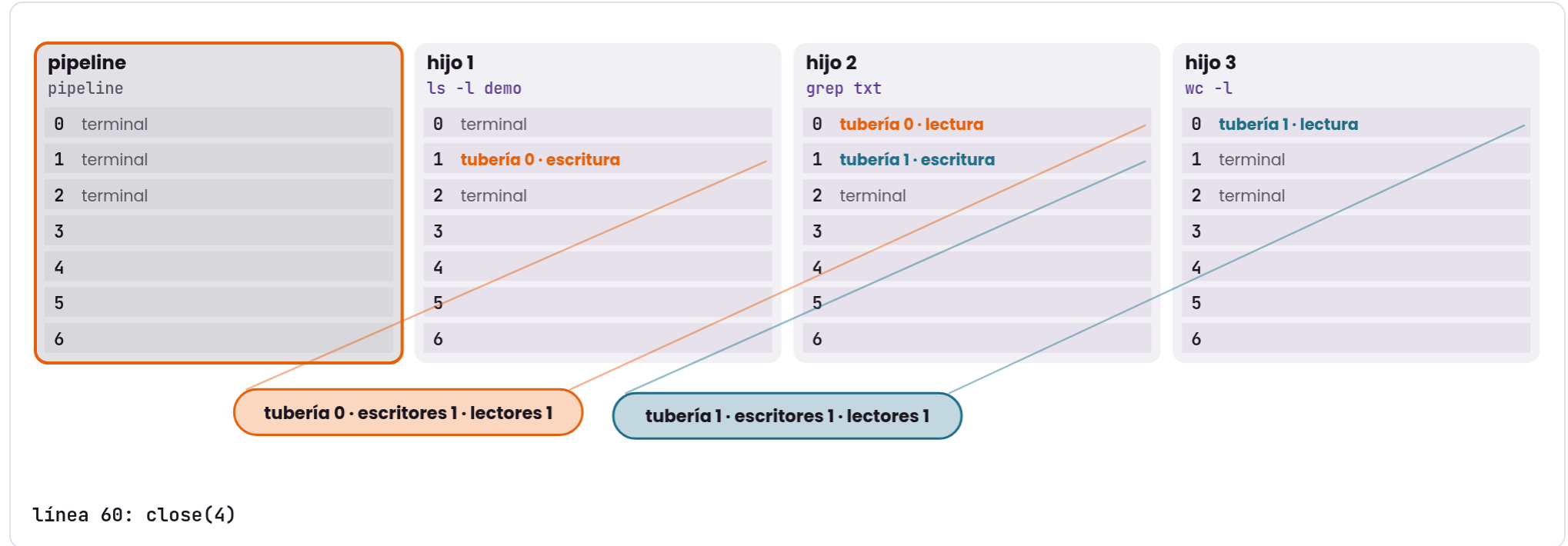
`ps` no sabe nada de tuberías: escribe en su descriptor 1, que `dup2` conectó a la tubería. El hijo lee hasta el fin de archivo y convierte a mayúsculas.

Lo que hace el shell con

```
ls -l | tr -d '[:blank:]'
```

1. `pipe(fds)`
2. `fork` → hijo 1: `dup2(fds[1], 1)`, cierra ambos, `exec("ls")`
3. `fork` → hijo 2: `dup2(fds[0], 0)`, cierra ambos, `exec("tr")`
4. El shell cierra sus dos extremos y espera a los dos hijos

Las tablas de descriptores, paso a paso



N comandos y el estado de la tubería

C tds/td4/pipeline_args.c

```
if (pid == 0) {
    if (in != STDIN_FILENO) {
        exit_if(dup2(in, STDIN_FILENO) == -1, "dup2");
        exit_if(close(in) == -1, "close");
    }
    if (fds[1] != -1) {
        exit_if(dup2(fds[1], STDOUT_FILENO) == -1,
            exit_if(close(fds[0]) == -1 || close(fds[1])
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
}
```

Salida real

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Salida real

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

Salida real

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Como el shell, devuelve el estado del **último** comando (`set -o pipefail` cambia esa regla en bash).

Más allá de `pipe`

`pipe2`

`O_CLOEXEC` evita que la tubería se filtre a programas lanzados con `exec`; `O_NONBLOCK` para no bloquear.

`splice`

Mueve datos entre tubería y archivo o socket sin copiarlos al proceso.

Capacidad

`fcntl(F_SETPIPE_SZ)` la agranda hasta `/proc/sys/fs/pipe-max-size`.

TD4 resuelto + desafío con IA

- `mypipe` con 10, 300 y 10 000 mensajes; con y sin el error
- `upper : ps` en mayúsculas
- `pipeline_args ls -l -- grep txt -- wc -l`

Bash

```
make -C tds && tds/tests/run.sh td4
```



Universidad
Indoamérica

PARA LLEVAR

Tres preguntas

1. ¿Cuándo devuelve 0 un `read` sobre una tubería?
2. ¿Por qué `mypipe 10000` termina con 141 y `mypipe_bug` no termina?
3. En `a | b | c`, ¿cuántos `pipe`, `fork` y `close` hace el shell?